

**Supplementary Material for
Residual-Guided Hypothesis Language Induction for Scientific Reasoning with
Small Language Models**

Anonymous submission

Appendix: Experimental Details and Ablations

This appendix is deliberately more explicit than the main text. The main paper reports compact benchmark-level claims; the appendix separates four questions: which complete runs support the headline table, how external reasoning controls are separated from RG-HLI variants, which components account for the DiscoveryBench gain, and which diagnostic deltas remain ambiguous.

Table 1: Complete-run audit used for the headline results. Each row uses the benchmark-native evaluator and preserves the corresponding run artifacts.

Benchmark	Tasks	Core model	Evaluator / judge	Benchmark result	Retained evidence
DiscoveryBench	239	GPT-4o-mini	official real-test HMS; GPT-4o judge	28.09 HMS; 36.04 C-HMS	predictions, official eval JSONL, summary
NewtonBench	324	GPT-4o-mini	official symbolic-law judge; audited regressions	49.7% SA-all; 67.1% SA-answered	row CSV, raw summary, audited summary
UltraHorizon	96	GPT-4o-mini	paper-style environment score	75.36 full-system score; 51.04 strict-agent score	per-run environment outputs, strict/full summaries

External Reasoning Baselines

Matched non-RG-HLI baselines are useful only when they are run on the same complete benchmark set. Table 2 reports the completed DiscoveryBench controls that satisfy this condition. They use the same GPT-4o-mini core, the same 239 real-test tasks, and the same HMS evaluator as the headline DiscoveryBench run, but they do not expose RG-HLI typed state, residual objects, answer-slot closure, or operator promotion.

Table 2: Matched external reasoning controls on DiscoveryBench. All rows are complete 239-task runs scored by the same official HMS evaluator.

Method	Tasks	HMS	Interface available to the model
Direct GPT-4o-mini	239	9.54	table snapshots and metadata in one prompt
REACT GPT-4o-mini	239	11.68	explicit thought/check/observation trace over the same context
CodeAct GPT-4o-mini	239	12.31	executed pandas summaries and one-pass answer rendering
RG-HLI GPT-4o-mini	239	28.09	typed contracts, executable probes, residual state, and renderers

The gap is not evidence that ordinary prompting is useless. It shows that, under the DiscoveryBench HMS contract, longer traces and one-pass executed summaries do not reproduce the effect of typed answer construction and residual-conditioned rendering.

Table 3: Reserved matched backbone-scaling matrix for RG-HLI. Each filled row must use the same frozen RG-HLI language, benchmark split, evaluator, and budget as the corresponding headline GPT-4o-mini run. Blank cells are reserved for future complete runs, not for slice or smoke results.

Benchmark	RG-HLI/GPT-4o-mini	RG-HLI/GPT-4o	RG-HLI/DeepSeek	Metric
DiscoveryBench	28.09	—	—	HMS, full239
NewtonBench	49.7	—	—	SA-all, full324
UltraHorizon	75.36	—	—	paper-style score, full96

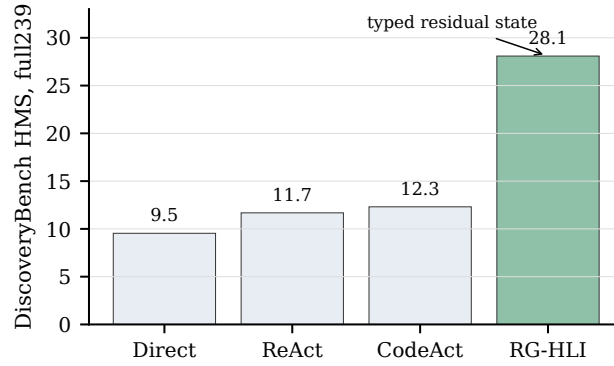


Figure 1: Matched external controls on DiscoveryBench. All bars are complete 239-task runs with the same GPT-4o-mini core and official HMS evaluator.

Component Ablations

Table 4: Full-run component ablations with interpretation. All rows are complete benchmark runs or complete official grids.

Benchmark	Condition	Tasks	Score	Interpretation
DiscoveryBench	fixed-language-none	239	0.68 HMS	removes typed Discovery contracts and renderers
DiscoveryBench	RG-HLI full	239	28.09 HMS / 36.04 C-HMS	full typed contract, validation, and residual stack
NewtonBench	chart-diagnostic control	324	47.5% SA-all / 64.7% SA-answered	residual-born chart recombination removed
NewtonBench	promotion-gate-off	324	49.4% SA-all / 66.7% SA-answered	held-out promotion gate disabled
NewtonBench	consistency-audited v9	324	49.7% SA-all / 67.1% SA-answered	final audited Newton run
UltraHorizon	all-off strict	96	0.00	induction and protocol support disabled
UltraHorizon	strict agent-only	96	51.04	no terminal protocol controller
UltraHorizon	full RG-HLI	96	75.36	public terminal protocol controller enabled

Table 5: Diagnostic deltas derived from Table 4. Positive values indicate that the second condition scores higher than the first under the listed metric.

Benchmark	Comparison	Delta	Reading
DiscoveryBench	fixed-language-none → full RG-HLI	+27.41 HMS	typed contracts and rendering are necessary on the full set
NewtonBench	chart-diagnostic control → audited RG-HLI	+2.20 SA-all	final gain is not explained by chart recombination alone
NewtonBench	chart-diagnostic control → audited RG-HLI	+2.40 SA-answered	same diagnostic on answered tasks
NewtonBench	promotion-gate-off → audited RG-HLI	+0.30 SA-all	promotion gating is useful but not the sole Newton driver
NewtonBench	promotion-gate-off → audited RG-HLI	+0.40 SA-answered	same diagnostic on answered tasks
UltraHorizon	all-off strict → strict agent-only	+51.04	executable induction and validation recover nontrivial reasoning
UltraHorizon	strict agent-only → full RG-HLI	+24.32	terminal protocol control converts induced evidence into valid submissions

Failure Modes by Benchmark

Table 6: Residual types exposed by the complete runs. These categories guide future operator induction; they are not separate benchmark-specific solvers.

Benchmark	Dominant residual type	Practical consequence
DiscoveryBench	semantic and structural residuals: wrong table, population, denominator, or answer facet	measured evidence can be useful but rendered in the wrong scientific facet
NewtonBench	algebraic residuals: missing coordinate transforms, constants, exponents, or unit-compatible charts	executable checking works after the right representation appears, but abstentions remain
UltraHorizon	temporal and protocol residuals: failed replay, local-only invariants, missing exploration evidence, unmet terminal preconditions	sequence is strong; Bio needs terminal contract control; grid remains limited by state abstraction and exploration

The DiscoveryBench ablation is the clearest positive component test: single-stage probes, the hypothesis workbench, and slot contracts alone are insufficient, while answer-slot closure and the full evidence-contract stack produce large gains. The NewtonBench ablations have a narrower diagnostic meaning. Removing residual-born operator charts lowers the full grid, while turning off the promotion gate leaves a near-headline score. The reported Newton result should therefore not be attributed to one benchmark-specific chart library or one promotion threshold; it is carried by the shared active-probe, residual representation, and symbolic-verification kernel. UltraHorizon adds a different diagnostic: the strict agent-only row shows that hypothesis induction can be present while official terminal conditions remain unmet. The full-system UltraHorizon result should therefore be read as the score of the complete RG-HLI pipeline, not as a raw-agent score.

Protocol Controls

Table 7 lists the controls used to keep the reported system as a single hypothesis-induction method. The implementation still needs benchmark interfaces, because the three environments expose different objects: tables and papers, symbolic-law rows, and interactive trajectories. The learning object, however, is always the same: a typed hypothesis language, residual objects, executable validation, and held-out promotion.

Table 7: Controls separating the universal method from benchmark-specific engineering. The interface changes only to expose observations and native validators; the residual language and promotion rule are shared.

Control	Fixed across benchmarks	Why it matters
Core model	GPT-4o-mini for the headline RG-HLI runs	isolates whether the external hypothesis-language layer helps a small model
Hypothesis state	typed artifacts, holes, contracts, validators, residuals	prevents the method from becoming only longer prompting
Failure signal	structured residual object rather than free-form critique	makes errors clusterable and promotable across future tasks
Promotion rule	held-out gain minus complexity penalty	limits memorization and uncontrolled library growth
Evaluator	benchmark-native judge or environment score	keeps reported numbers in the metric used by each benchmark

Table 8: How the same residual-to-operator loop appears in the three benchmark families. These are not separate solvers; they are domain-specific observations fed into the same state machine.

Benchmark	Observation exposed to the kernel	Residual object	Promotable operator class
DiscoveryBench	tables, text snippets, requested scientific claim	missing role, wrong denominator, wrong answer facet	answer-slot closure, evidence renderer, consistency check
NewtonBench	numerical rows, candidate variables, target expression	wrong exponent, constant, transform, or abstention	executable probe, symbolic fit, unit-compatible verifier
UltraHorizon	trajectory, action log, rule feedback, replay state	failed replay, local invariant, missing exploration evidence	exploration policy, replay validator, temporal abstraction

Prompt and Operator Interface

The benchmark-facing prompts are intentionally narrow. The model is not asked to “think longer” or to invent a complete answer in free text. Each call must return one typed object that can be executed, checked, or converted into a residual. Table 9 gives

the prompt contracts used by the kernel. The same contract structure is used across all three benchmarks; only the observation serializer and native validator differ.

Table 9: Prompt contracts used by RG-HLI. Each prompt returns a typed object with an attached validation path rather than an unconstrained chain of thought.

Call	Input state	Required output	Rejection rule
Contract compiler	task observation, available tools, current L^*	roles, admissible answer form, measurable holes, validator names	missing target role or non-executable hole
Probe writer	contract, hole, evidence table or trajectory	short executable measurement with declared inputs and expected type	code fails, reads forbidden fields, or returns wrong type
Hypothesis composer	closed holes, evidence objects, active operators	artifact h : law, claim, program, or rule mapping	artifact cannot be replayed or violates contract
Residual compiler	failed validation trace	typed residual $(\tau, \ell, c, e, v, \eta)$	free-text criticism without location, evidence, and failed validator
Operator proposer	recurring residual class R_k and held-out tasks	candidate operator o^* with scope, cost, and validator hooks	no residual class closed, no executable hook, or excessive complexity

Operators are added through a single interface. The implementation may contain benchmark adapters, but an accepted operator must expose the same fields: scope, precondition, executable transform, evidence update, residual update, complexity cost, and promotion evidence. In simplified form, an operator is accepted only if it satisfies the following contract:

```
operator.id: stable name;
operator.scope: residual types and artifact slots where it is admissible;
operator.precondition(state, contract) -> bool;
operator.apply(state, contract) -> artifact/evidence/residual update;
operator.cost -> nonnegative complexity;
operator.validators -> executable checks that must pass;
promote iff heldout_gain(operator) > lambda * operator.cost.
```

This interface is the main difference between residual-conditioned language growth and ordinary prompt iteration. A failed hypothesis does not merely produce another instruction for the model. It produces a typed residual with a location and failed validator; only repeated residuals can ask for a new operator, and the operator is not retained unless it improves held-out tasks under the same validator stack.

Worked Hypothesis Construction Trace

Table 10 shows the intended granularity of a single DiscoveryBench-style task. The model is not asked to guess the final scientific sentence in one pass. It first commits to a typed contract, then executable measurements close slots, and only the rendered artifact is sent to the native evaluator. A failure is not stored as prose feedback; it is stored as the violated slot or validator.

Table 10: A per-task trace of typed hypothesis construction. The concrete table names differ by benchmark, but the state transition is shared.

Stage	Object committed to state	Check performed
Task observation	query, tables, metadata, admissible answer form	no gold label or judge feedback is available
Evidence contract	roles, target slot, measurable holes, rendering constraints	every hole must have a declared data source
Executable closure	small probes estimate denominators, groups, trends, or constants	code returns typed evidence rather than prose justification
Artifact rendering	hypothesis text plus workflow grounded in closed slots	native evaluator sees only the final artifact
Residualization	failed slot, location, evidence, validator, environment	recurring residuals can induce an operator after held-out promotion

Qualitative Operator-Induction Example

The following DiscoveryBench example illustrates the intended language-growth mechanism. A first full audit exposed repeated failures on study-design tables: the system often measured a real effect but selected the wrong role, such as confusing original and replication studies or measuring the complement of a binary design variable. These errors produced a shared residual type: *study-design role closure*. The induced operator did not memorize a dataset answer. It added a reusable repair interface: conservative aliases for original/replication roles, paired-column completion, binary complement closure, and contrastive rendering of the selected role.

Table 11: Example of a residual-induced operator. The operator is described as a repair interface over typed residuals, not as a benchmark answer template.

Element	Description
Observed failures	repeated wrong-role and wrong-complement answers in study-design tables
Residual object	(τ_ρ = role-closure, ℓ_ρ = answer slot, c_ρ = study role, e_ρ = paired columns, v_ρ = consistency check)
Candidate operator	close original/replication roles with conservative aliases, paired-column completion, binary complements, and contrastive rendering
Promotion evidence	the operator must pass held-out promotion before inclusion in the frozen language
Known regressions	plant introduction and some NLS cases regress because denominator alignment and semantic rendering remain separate residuals

Leakage and Freeze Controls

The paper uses two distinct phases. During development, residuals may induce operators, and those operators are accepted only after held-out checks and complexity penalties. During the reported benchmark evaluations, the language is frozen: the system may search within L^* for each task, but it does not use the task’s benchmark score to update L^* for later reported tasks. Table 12 summarizes the protocol.

Table 12: Protocol controls for residual-conditioned language growth.

Phase	Allowed	Disallowed
Development induction	collect residuals, propose operators, held-out promotion	using final reported test scores as promotion targets
Headline evaluation	per-task search, execution, validation, rendering in frozen L^*	changing L^* from future test rows or gold labels
Post-run analysis	cluster failures and plan future operators	retroactively changing the reported score

Compute Receipts

The artifact set preserves row-level outputs for the headline runs and full-run ablations. Because the three benchmark harnesses differ in native tool use, environment runtime, and judge calls, compute is reported at the harness level rather than collapsed into a single cross-benchmark token number.

Table 13: Available full-run artifact receipts. Each row points to a retained row-level output used for the corresponding table entry; the short receipt IDs map directly to experiment directories in the released artifact tree.

Benchmark	Tasks	Score	Artifact
DiscoveryBench fixed-language-none	239	0.68 HMS	DB-none-full1239
DiscoveryBench Direct GPT-4o-mini	239	9.54 HMS	DB-direct-full1239
DiscoveryBench ReAct GPT-4o-mini	239	11.68 HMS	DB-react-full1239
DiscoveryBench CodeAct GPT-4o-mini	239	12.31 HMS	DB-codeact-full1239
DiscoveryBench full RG-HLI	239	28.09 HMS	DB-RG-HLI-full1239
NewtonBench chart-control	324	47.5% SA-all	NB-chart-control-full1324
NewtonBench promotion-gate-off	324	49.4% SA-all	NB-no-gate-full1324
NewtonBench full RG-HLI	324	49.7% SA-all	NB-RG-HLI-full1324
UltraHorizon strict agent-only	96	51.04	UH-strict-full196
UltraHorizon full RG-HLI	96	75.36	UH-full-system-full196